Région Hauts-de-France

Cofinancé par l'Union européenne

realconseil
DIAGNOSTIC ACCOMPAGNEMENT FORMATION

SYMFONY 6 – La sécurité User et hashage de mot de pass – Part2

Introduction

Notre User est fait avec les constraints, nous avons vu le security.yaml nous avons généré les fixtures en hashant les mots de passe. Nous allons voir une autre façon de faire pour hasher les mots de passe de façon moins redondante dans ce chapitre

I) Les Entity Listeners

Ce sont des fichiers qui vont écouter ce qui se passe au niveau des entity et faire plusieurs actions au niveau de la peristence, mise à jour etc

- Commençons donc par supprimer ce que nous avons fait dans Appfixture à ce niveau

```
69                  ->setEmail($this->faker->email())
70                  ->setRoles(['ROLE_USER']);
71
72              $hashPassword = $this->hasher->hashPassword(
73                  $user,
74                  'password'
75              );
76              $user->setPassword($hashPassword);
77
78
79              $manager->persist($user);
80          }
81
```

A EFFACER

- C'est là que va intervenir plainPassword ajoutons le comme ceci

```
66              $user = new User();
67              $user -> setName($this->faker->name())
68                  ->setPseudo(mt_rand(0,1)===1 ? $this->faker->
69                  ->setEmail($this->faker->email())
70                  ->setRoles(['ROLE_USER'])
71                  ->setPlainPassword("password");
72
73              $manager->persist($user);
74          }
75
```

A AJOUTER

- Allons dans notre dossier config et dans le fichier service.yaml

Nous allons faire intervenir notre entityListener pour que ça match la ressource src EntitListener

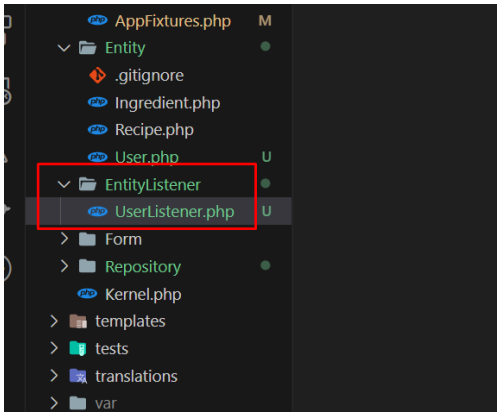
Ajoutre un tag qui es « doctrine.orm.entity_listener »

```
18          exclude:
19              - '../src/DependencyInjection/'
20              - '../src/Entity/'
21              - '../src/Kernel.php'
22      App\EntityListener\:
23          resource: "../src/EntityListener/"
24          tags: ["doctrine.orm.entity_listener"]
25
26
```

- Sur l'entity User nous allons ajouter le décorateur de l' « EntityListener » comme ci-dessous

```
9   use Symfony\Component\Validator\Constraints as Assert;
10  use Symfony\Bridge\Doctrine\Validator\Constraints\UniqueEntity;
11
12  #[UniqueEntity('email')]
13  #[ORM\Entity(repositoryClass: UserRepository::class)]
14  #[ORM\EntityListeners('App\EntityListener\UserListener')]
15  class User implements UserInterface, PasswordAuthenticatedUserInterface
16  {
17      #[ORM\Id]
```

- Création du dossier EntityListener dans le dossier src et dans ce dossier un fichier userListener.php



Dans ce fichier nous allons procéder à notre encodage

- On fait le construct et on crée avant la variable $hasher en private

```
1   <?php
2
3   namespace App\EntityListener;
4
5   use App\Entity\User;
6   use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;
7
8   class UserListener
9   {
10      private UserPasswordHasherInterface $hasher;
11
12      public function __construct(UserPasswordHasherInterface $hasher)
13      {
14          $this->hasher = $hasher;
15      }
```

- On crée une public function prepersist

```
15      }
16
17      public function prePersist(User $user)
18      {
19          $this->encodePassword($user);
20      }
21
22      public function preUpdate(User $user)
```

- On crée une public function preUpdate

```
        {
            $this->encodePassword($user);
        }

        public function preUpdate(User $user)
        {
            $this->encodePassword($user);
        }

        /**
         * Encode password based on plain password
```

- Nous créons la publicFunction où tout se passe dedans pour l'encodage que l'on appellera encode()

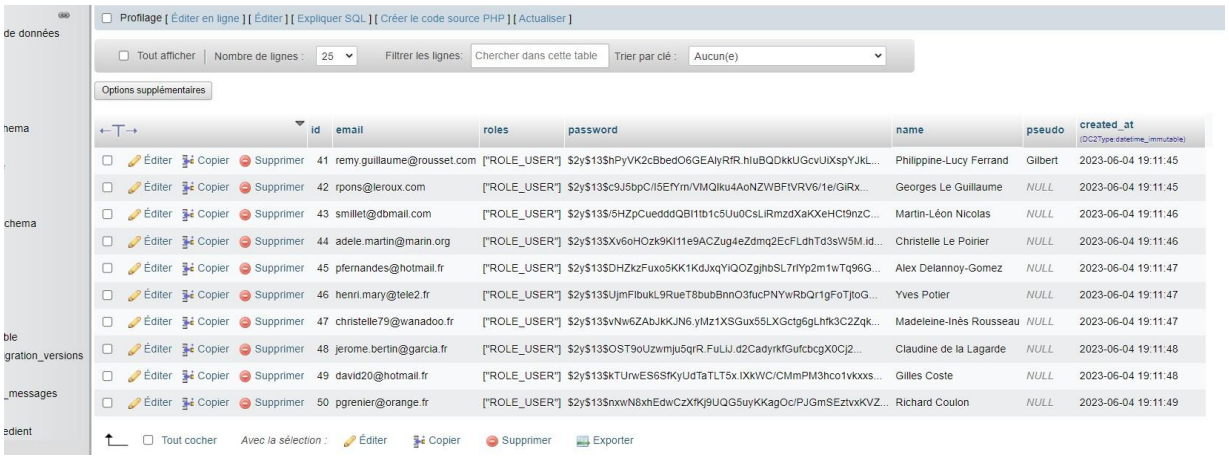
```
22      public function preUpdate(User $user)
23      {
24          $this->encodePassword($user);
25      }
26
27      /**
28       * Encode password based on plain password
29       *
30       * @param User $user
31       * @return void
32       */
33      public function encodePassword(User $user)
34      {
35          if($user->getPlainPassword()===null){
36              return;
37          }
38          $user->setPassword(
39              $this->hasher->hashPassword(
40                  $user,
41                  $user->getPlainPassword()
42              )
43              );
44      }
45  }
```

Grâce à ce fichier nous allons encoder notre plainPassword que nous avons dans notre appFixture car nous avons mis le décorateur sur notre entity user et nous avons lié cet Entitylistener sur le User

On ajoute pour finir au $user un setPlainPassword que l'on passe à null pour être certain de ne pas avoir de soucis comme ceci :

```
40                $user,
41                $user->getPlainPassword()
42            )
43        );
44        $user->setPlainPassword(null);
45    }
```

Envoyons à nouveau nos données en bdd pour tester si tout cela fonctionne bien avec la commande « php bin/console doctrine:fixtures:load »



Tout s'est très bien passé notre EntityListener fonctionne très bien, et c'est de bonne pratique de procéder de cette façon.

Dans le prochain chapitre nous allons voir le firewall et la sécurité du login